

# Calcolo Combinatorio

| MODELLO                   | Cosa faccio?       | Ordine conta?   | Ripetizione Elementi? | Esempio "Vita Reale"       | Struttura Dati Chiave                                      |
|---------------------------|--------------------|-----------------|-----------------------|----------------------------|------------------------------------------------------------|
| 1. Combinazioni Semplici  | Scelgo K           | NO              | NO                    | SuperEnalotto              | <code>idx + 1</code> nella ricorsione                      |
| 2. Combinazioni con Rip.  | Scelgo K           | NO              | Sì                    | Gusti Gelato               | <code>idx</code> nella ricorsione (sto fermo)              |
| 3. Permutazioni Semplici  | Ordino Tutti       | Sì              | NO                    | Anagramma "ROMA"           | <code>bool visited[]</code>                                |
| 4. Permutazioni con Rip.  | Ordino Tutti       | Sì              | Sì (Input doppio)     | Anagramma "ANNA"           | <code>sort + if(v[i]==v[i-1]) skip</code>                  |
| 5. Disposizioni Semplici  | Scelgo K e Ordino  | Sì              | NO                    | Podio Gara (Oro, Arg, Bro) | <code>visited[]</code> + stop a profondità K               |
| 6. Disposizioni con Rip.  | Scelgo K e Ordino  | Sì              | Sì                    | PIN Bancomat / Password    | Loop sempre da <code>0</code> a <code>N</code>             |
| 7. Insieme Parti (Subset) | Prendo/Lascio      | NO              | NO                    | Zaino (Knapsack)           | Ramo <code>prendo</code> ,<br>Ramo <code>non prendo</code> |
| 8. Partizioni (Secchi)    | Distribuisco Tutti | NO (nei gruppi) | NO (Input unico)      | Squadre di Calcio          | Array <code>bucket_sums[]</code>                           |
| 9. Clique (Grafo)         | Cerco Gruppo Denso | NO              | NO                    | Amici su Facebook          | Matrice Adiacenza ( <code>adj[i][j]</code> )               |

# QUANTI modi ci sono?

## 1. COMBINAZIONI (L'Ordine NON conta)

*Sto creando un sottoinsieme di dimensione K.  $\{A, B\}$  è uguale a  $\{B, A\}$ .*

### A. Combinazioni Semplici (No Ripetizioni)

- **Cos'è:** Pesco K oggetti diversi da N disponibili. Una volta preso, non posso riprenderlo.
- **Esempio:** SuperEnalotto, Mano di Poker (5 carte dal mazzo).
- **Logica Code:** Ho un indice `i`. Nella ricorsione passo `i + 1` (passo al prossimo elemento).

```
// Chiamata ricorsiva  
solve(idx + 1, ...); // Scarto l'elemento corrente o passo al prossimo
```

### B. Combinazioni con Ripetizione (Multiset)

- **Cos'è:** Pesco K oggetti da N *tipi* disponibili. Posso prendere lo stesso tipo più volte.
- **Esempio:** Il Gelataio. Ho 3 gusti (Cioccolato, Fragola, Limone) e voglio una coppetta da 2 palline. Posso fare "Cioccolato-Cioccolato".
- **Esempio Numerico:** Trovare 3 numeri che sommano a N, potendo usare lo stesso numero più volte (es:  $2+2+2 = 6$ ).
- **Logica Code:** Ho un indice `i`. Nella ricorsione passo `i` (rimango sullo stesso elemento perché posso riprenderlo).

```
// Chiamata ricorsiva  
solve(idx, ...);  
// Rimango su 'idx' per poterlo ripescare
```

## 2. DISPOSIZIONI (L'Ordine CONTA, Sottoinsieme)

*Scelgo K elementi su N.  $\{A, B\}$  è diverso da  $\{B, A\}$ .*

### A. Disposizioni Semplici

- **Cos'è:** Assegno ruoli o posizioni diverse a oggetti distinti.
- **Esempio:** Podio di una gara (Oro, Argento, Bronzo) con 10 atleti.
- **Logica Code:** Uso un array `visited[]`. Se `visited[i] == true`, salto. Altrimenti prendo, segno true, ricorro, segno false (backtrack).

### B. Disposizioni con Ripetizione

- **Cos'è:** Stringhe, Password, Codici. Ogni posizione è indipendente dalle altre.

- **Esempio:** Aprire un lucchetto a 3 cifre (0-9). La combinazione 9-9-9 è valida.
- **Esempio Matematico:** Prodotto Cartesiano.
- **Logica Code:** Il ciclo `for` riparte sempre da 0 (o dall'inizio delle opzioni) a ogni livello della ricorsione. Non c'è `visited`.

```
for (int i = 0; i < N; i++) {
    sol[level] = opzioni[i]; // Prendo l'opzione
    solve(level + 1); // Scendo di livello
}
```

### 3. PERMUTAZIONI (L'Ordine CONTA, Tutto l'insieme)

*Rimescolo TUTTI gli N elementi. Dimensione fissa N.*

#### A. Permutazioni Semplici

- **Cos'è:** Anagramma di parola con lettere diverse (es. "ROMA").
- **Esempio:** Ordine di visita di 5 città nel Commesso Viaggiatore.
- **Logica Code:** Identica alle Disposizioni Semplici (`visited[]`), ma il caso base è quando `count == N`.

#### B. Permutazioni con Ripetizione (Elementi indistinguibili)

- **Cos'è:** Anagramma di parola con lettere doppie (es. "TETTO"). Se scambio le due 'T', la parola non cambia, quindi non devo contarla due volte.
- **Esempio:** Ho 2 bandiere rosse e 1 blu. In che ordine posso appenderle? (R-R-B, R-B-R, B-R-R).
- **Logica Code:**
  1. Si **ordina** l'array in input (fondamentale).
  2. Nel ciclo `for`, se l'elemento attuale è uguale al precedente (`v[i] == v[i-1]`) E il precedente non è stato usato in questo ramo, si fa `continue` (skip) per evitare duplicati identici.

### 4. INSIEME DELLE PARTI (Power Set)

*Sottoinsiemi di qualsiasi dimensione.*

#### A. Power Set Semplice

- **Cos'è:** Prendere o lasciare ogni elemento distinto.
- **Esempio:** Zaino (Knapsack) 0/1. Ogni oggetto è unico: o lo prendi o lo lasci.

#### B. Power Set con Ripetizione (Unbounded Knapsack)

- **Cos'è:** Hai oggetti con quantità infinita.
- **Esempio:** Zaino dove puoi prendere quante barrette di cioccolato vuoi (finché c'è spazio).
- **Logica Code:**
  - *Ramo 1 (Prendo):* Aggiungo valore, sottraggo peso, ma **rimango sullo stesso indice** (ricorsione `idx`).
  - *Ramo 2 (Lascio/Fine):* Passo all'indice successivo (`idx + 1`).

"COME posso raggrupparli?" / "QUAL È il gruppo più grande?"

## 5. PARTIZIONI (Il problema dei Secchi)

*Distribuire N oggetti in K gruppi.*

Qui la distinzione "Semplice vs Ripetizione" assume un significato diverso: **i gruppi sono distinguibili o no?**

### A. Partizioni di Insieme (Set Partitioning) - L'esercizio di prima

- **Cos'è:** Devi mettere ogni elemento in uno dei K secchi.
- **Ripetizione:**
  - **Degli Elementi: NO.** Ogni elemento va in UN solo secchio (Disgiunti).
  - **Dei Secchi: SÌ.** I secchi possono avere più elementi.
- **Esempio:** "Dividi gli studenti in 3 squadre di lavoro".
- **Codice Tipico (Backtracking):**
  - Scorro l'array degli elementi (`idx`).
  - Per ogni elemento, provo a metterlo nel Secchio 0, poi nel Secchio 1, etc.
  - **Constraint:** La somma del secchio non deve superare il target.

### B. Partizioni di un Intero (Integer Partition)

- **Cos'è:** Scrivere un numero come somma di altri numeri.
- **Ripetizione: SÌ.** Posso usare lo stesso numero più volte.
- **Esempio:** "In quanti modi posso scrivere 5?" (Es: `1+1+1+1+1`, `2+3`, `4+1`, `5`).
- **Differenza:** Qui non ho un array di input "oggetti". Ho solo un numero target.
- **Codice Tipico:** Simile al "Resto delle monete" (Unbounded Knapsack).

## 6. CLIQUE (Sottoinsiemi su Grafi)

*Trovare un gruppo di "amici" tutti collegati tra loro.*

La Clique è tecnicamente un problema di **Insieme delle Parti (Power Set)** con un vincolo fortissimo.

- **Cos'è:** Cerco un sottoinsieme di vertici  $V' \subseteq V$ .
- **Ripetizione: NO.** Un nodo o c'è o non c'è nel gruppo. Non posso avere "due volte Mario" nel gruppo di amici.
- **Logica Code:**
  1. È un algoritmo **Greedy/Backtracking**.
  2. Parto da un insieme vuoto o da un nodo.
  3. Provo ad aggiungere un nodo candidato.
  4. **CHECK (Il cuore della Clique):** Il nuovo nodo è collegato a **TUTTI** quelli già presenti nel gruppo?
    - Sì -> Aggiungo e ricorro.
    - No -> Scarto (Pruning immediato).